



华南理工大学
South China University of Technology

实验报告

课程名称：《数字信号处理实验》

学生姓名：顾昊瑜

学号：202364820071

专业班级：人工智能二班

开课学期：2024-2025 学年度第二学期

未来技术学院

2025 年 05 月

目录

1	实验目的	3
2	实验要求及内容	3
3	验证性实验分析与过程	4
3.1	交互式产生 5 种常见离散信号并绘图	4
3.2	比较 conv 和 filter 函数	7
3.3	使用 impz、filter、conv 函数求解以下两个系统:	9
4	验证性实验结果	13
4.1	五种离散信号的仿真实验结果	13
4.2	比较 conv 和 filter 函数	14
4.3	系统响应分析与验证	15
4.4	结果分析	17
4.5	实验结论	18
5	应用型实验: 语音基因周期估计	18
5.1	实验目的	18
5.2	实验原理	19
5.3	实验过程与代码分析	20
5.4	实验结果	23
5.5	结果分析	27
5.6	实验结论	30
6	应用型实验: 听力测试信号发生器	30
6.1	实验目的	30
6.2	频率与幅度调节原理	30
6.3	用户交互与界面设计	31
6.4	实验过程与代码分析	32
6.5	原理说明	34
6.6	实验结果	35
6.7	实验结论	35
7	实验心得	35
7.1	验证型实验心得	35
7.2	应用型实验心得	36
7.3	整体感悟与展望	36
8	致谢	36

数字信号产生及时域处理

地点: D3b-413

实验台号: 55

实验日期与时间: 2025.5.8

评分:

预习检查记录: /

实验教师: 宁更新

1 实验目的

1. 加深对常用离散信号的理解。
2. 熟悉使用 MATLAB 在时域中产生一些基本的离散时间信号。
3. 熟悉并掌握离散系统的差分方程表示法。
4. 加深对冲激响应和卷积分析方法的理解。
5. 设计一个听力测试信号发生器。
6. 估计一段语音的基音周期。
7. 设计一个视觉暂留时间的测试软件。

2 实验要求及内容

1. 编制程序产生单位抽样序列、单位阶跃序列、正弦序列、复指数序列、指数序列 5 种信号, 长度可输入确定, 函数需要的参数可输入确定, 并绘出其图形。
2. 以下程序中分别使用 conv 和 filter 函数计算 h 和 x 的卷积 y 和 y_1 , 运行程序, 并分析 y 和 y_1 是否有差别, 为什么要使用 $x[n]$ 补零后的 x_1 来产生 y_1 ; 具体分析当 $h[n]$ 有 i 个值, $x[n]$ 有 j 个值, 使用 filter 完成卷积功能, 需要如何补零?
3. 编制程序求解下列两个系统的单位冲激响应和阶跃响应, 并绘出其图形。要求分别用 filter、conv、impz 三种函数完成。

$$y[n] + 0.75y[n-1] + 0.125y[n-2] = x[n] - x[n-1],$$

$$y[n] = 0.25 \{x[n-1] + x[n-2] + x[n-3] + x[n-4]\},$$

3 验证性实验分析与过程

3.1 交互式产生 5 种常见离散信号并绘图

1. 单位抽样序列

单位抽样序列定义如下:

$$\delta(n) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases}$$

在 MATLAB 中可以利用 `zeros()` 函数实现:

$$x = [1 \quad \text{zeros}(1, N - 1)]$$

2. 单位阶跃序列

单位阶跃序列定义如下:

$$u(n) = \begin{cases} 1, & n \geq 0 \\ 0, & n < 0 \end{cases}$$

在 MATLAB 中可以利用 `ones()` 函数实现:

$$x = \text{ones}(1, N);$$

3. 正弦序列

正弦序列定义如下:

$$x(n) = A \sin\left(2\pi f \frac{n}{F_s} + \varphi\right)$$

在 MATLAB 中实现过程如下:

```
n = 0:N-1;  
x = A * sin(2 * pi * f * n / Fs + fai);
```

4. 复指数序列

复指数序列定义如下:

$$x(n) = r \cdot e^{j\omega n}$$

在 MATLAB 中实现过程如下:

```
n = 0:N-1;  
x = r .* exp(1j * w * n);
```

5. 指数序列

指数序列定义如下:

$$x(n) = a^n$$

在 MATLAB 中实现过程如下:

```
n = 0:N-1;  
x = a.^n;
```

3.1.1 绘图说明

上述信号可以利用 MATLAB 的 stem() 或 plot() 函数进行绘图。例如, 可以使用以下语句将其以离散信号形式绘制:

```
stem(n, x);  
xlabel('n');  
ylabel('x(n)');  
title('离散信号 x(n)');
```

绘制代码如下:

Listing 1: 交互式产生 5 种常见离散信号 (参数输入与信号生成)

```
clear; clc;

%% 用户输入部分
N = input('请输入信号长度 N (正整数) : ');
n = 0:N-1;

fprintf('\n-- 正弦信号参数输入 --\n');
A = input(' 振幅 A = ');
f = input(' 频率 f (Hz) = ');
Fs = input(' 采样频率 Fs (Hz) = ');
phi = input(' 相位 phi (rad) = ');

fprintf('\n-- 复指数信号参数输入 --\n');
r = input(' 幅值 r = ');
w = input(' 角频率 w (rad/sample) = ');

fprintf('\n-- 指数序列参数输入 --\n');
a = input(' 底数 a = ');

%% 信号生成
Δ = zeros(1, N); Δ(1) = 1;
u = ones(1, N);
x_sin = A * sin(2*pi * f * n / Fs + phi);
x_cexp = r * exp(1j * w * n);
x_exp = a .^ n;
```

Listing 2: 交互式产生 5 种常见离散信号 (绘图部分)

```

figure('Name','五种离散信号','NumberTitle','off','Position',[100 100 600 ...
    800]);

subplot(5,1,1);
stem(n,  $\Delta$ , 'filled');
grid on;
title('1. 单位抽样序列  $\Delta[n]$ ');
xlabel('n'); ylabel('\Delta[n]');

subplot(5,1,2);
stem(n, u, 'filled');
grid on;
title('2. 单位阶跃序列  $u[n]$ ');
xlabel('n'); ylabel('u[n]');

subplot(5,1,3);
plot(n, x_sin, 'o-', 'Linewidth',1);
grid on;
title(sprintf('3. 正弦序列: A=%.2g, f=%.2gHz, Fs=%.2gHz, \phi=%.2g', ...
    A, f, Fs, phi));
xlabel('n'); ylabel('x_{sin}[n]');

subplot(5,1,4);
plot(n, real(x_cexp), 'o-', n, imag(x_cexp), 's--', 'Linewidth',1);
grid on;
legend('Re\{x_{cexp}\}', 'Im\{x_{cexp}\}', 'Location', 'best');
title(sprintf('4. 复指数序列: r=%.2g, \omega=%.2g rad/sample', r, w));
xlabel('n'); ylabel('幅度');

subplot(5,1,5);
plot(n, x_exp, '.-', 'Linewidth',1);
grid on;
title(sprintf('5. 指数序列: a=%.2g', a));
xlabel('n'); ylabel('x_{exp}[n]');

tightfig;

```

3.2 比较 conv 和 filter 函数

考虑以下 MATLAB 程序片段:


```
h = [1 2 3]; % 系统响应 h[n], 长度为 i = 3
x = [1 2 3 4]; % 输入序列 x[n], 长度为 j = 4

y = conv(h, x); % 卷积结果
x1 = [x, zeros(1, length(h)-1)]; % 对 x 补零
y1 = filter(h, 1, x1); % 滤波器计算结果
```

y 和 y1 是否有差别？

在本例中， y 和 y_1 结果是**相同的**。这是因为对 $x[n]$ 补零（长度增加 $i - 1$ ）后使用 filter 实际上完成了与 conv 等价的线性卷积操作。

为什么使用 filter 时要对 $x[n]$ 补零？

MATLAB 中 filter(b,a,x) 函数实现的是线性差分方程的**递归计算**，适用于 IIR/FIR 滤波器，等价于系统的差分方程求解。对于 FIR 系统，即 $a = 1$ 的情形，filter 执行的是：

$$y[n] = \sum_{k=0}^{i-1} h[k] \cdot x[n-k]$$

由于系统具有记忆长度 i ，如果不补零，filter 无法完整覆盖所有 n 对应的输出样本，而 conv 默认会输出长度为 $i + j - 1$ 的完整线性卷积结果。

因此，要让 filter 与 conv 结果一致，需将 $x[n]$ 补零至长度 $j + i - 1$ ，使输出完整。

总结

若 $h[n]$ 长度为 i ， $x[n]$ 长度为 j ，则：
conv(h, x) 返回长度为 $i + j - 1$ 的完整卷积。为使 filter(h, 1, x1) 与 conv 等效，应补零 $i - 1$ 项，使 $x1$ 长度为 $j + i - 1$ 。

Listing 3: 比较 conv 和 filter 函数

```

clf;
h = [3 2 1 -2 1 0 -4 0 3]; % impulse response
x = [1 -2 3 -4 3 2 1]; % input sequence

y = conv(h, x); % 用卷积求输出
n = 0:14;

subplot(2,1,1);
stem(n, y);
xlabel('Time index n');
ylabel('Amplitude');
title('Output Obtained by Convolution');
grid;

x1 = [x zeros(1,8)]; % 零补齐输入信号
y1 = filter(h, 1, x1); % 用 filter 求输出

subplot(2,1,2);
stem(n, y1);
xlabel('Time index n');
ylabel('Amplitude');
title('Output Generated by Filtering');
grid;

```

3.3 使用 impz、filter、conv 函数求解以下两个系统:

系统 1:

$$y[n] + 0.75y[n-1] + 0.125y[n-2] = x[n] - x[n-1],$$

3.3.1 系统定义与信号初始化

Listing 4: 初始化系统参数和激励信号

```

clear; close all; clc;

N = 40; % 样本点数
n = 0:N-1; % 横轴索引
Δ = [1 zeros(1,N-1)]; % 单位冲激信号 δ[n]
u = ones(1,N); % 单位阶跃信号 u[n]

b1 = [1 -1]; % 分子系数
a1 = [1 0.75 0.125]; % 分母系数

```

3.3.2 使用 filter 函数求解响应

Listing 5: 使用 filter 函数求响应

```
h1_f = filter(b1, a1, Δ); % 冲激响应 h[n]  
s1_f = filter(b1, a1, u); % 阶跃响应 s[n]
```

3.3.3 使用 impz 和 cumsum

Listing 6: 用 impz 和 cumsum 计算响应

```
[h1_i, ~] = impz(b1, a1, N); % 冲激响应  
s1_i = cumsum(h1_i); % 阶跃响应 = 冲激响应累加
```

3.3.4 使用卷积 conv 方法

Listing 7: 使用 conv 函数求响应

```
h1_c = conv(h1_i, Δ); % 冲激响应 =  $\delta * h$   
h1_c = h1_c(1:N); % 取前 N 点  
  
s1_c = conv(h1_i, u); % 阶跃响应 =  $h * u$   
s1_c = s1_c(1:N); % 取前 N 点
```

3.3.5 可视化: 对比三种方法

Listing 8: 绘制子图

```
figure('Name','System 1 Responses','NumberTitle','off');
subplot(3,2,1); stem(n, h1_f, 'filled');
    title('h_1[n] by filter'); xlabel('n'); ylabel('h');
subplot(3,2,2); stem(n, h1_i, 'filled');
    title('h_1[n] by impz'); xlabel('n'); ylabel('h');
subplot(3,2,3); stem(n, h1_c, 'filled');
    title('h_1[n] by conv'); xlabel('n'); ylabel('h');

subplot(3,2,4); stem(n, s1_f, 'filled');
    title('s_1[n] by filter'); xlabel('n'); ylabel('s');
subplot(3,2,5); stem(n, s1_i, 'filled');
    title('s_1[n] by impz (cumsum)'); xlabel('n'); ylabel('s');
subplot(3,2,6); stem(n, s1_c, 'filled');
    title('s_1[n] by conv'); xlabel('n'); ylabel('s');
sgtitle('System 1: Impulse & Step Responses (3 methods)');
```

系统 2:

$$y[n] = 0.25 \{x[n-1] + x[n-2] + x[n-3] + x[n-4]\},$$

该系统为一个典型的长度为 4 的滑动平均 FIR 系统。

3.3.6 定义系统系数与初始信号

Listing 9: 定义系统参数 b2

```
b2 = 0.25*[0 1 1 1 1]; % 滑动平均滤波器 (延迟一位起)
a2 = 1;                % FIR 系统, 无反馈项
```

3.3.7 使用 filter 函数

Listing 10: 用 filter 函数求响应

```
h2_f = filter(b2, a2, Δ); % 冲激响应
s2_f = filter(b2, a2, u); % 阶跃响应
```

3.3.8 使用 `impz` + `cumsum` 方法Listing 11: 使用 `impz` 和 `cumsum` 求响应

```
[h2_i,~] = impz(b2, a2, N); % 冲激响应
s2_i = cumsum(h2_i); % 阶跃响应
```

3.3.9 使用 `conv` 卷积方法Listing 12: 使用 `conv` 函数进行卷积求响应

```
h2_c = conv(h2_i, Δ); %  $h[n] = \delta * h$ 
h2_c = h2_c(1:N); % 取前 N 项

s2_c = conv(h2_i, u); %  $s[n] = h * u$ 
s2_c = s2_c(1:N); % 取前 N 项
```

3.3.10 绘制结果: 3 种方法的 $h[n]$ 与 $s[n]$ 对比

Listing 13: 绘图展示三种方法下的信号

```
figure('Name','System 2 Responses','NumberTitle','off');

subplot(3,2,1); stem(n, h2_f,'filled');
    title('h_2[n] by filter'); xlabel('n'); ylabel('h');
subplot(3,2,2); stem(n, h2_i,'filled');
    title('h_2[n] by impz'); xlabel('n'); ylabel('h');
subplot(3,2,3); stem(n, h2_c,'filled');
    title('h_2[n] by conv'); xlabel('n'); ylabel('h');

subplot(3,2,4); stem(n, s2_f,'filled');
    title('s_2[n] by filter'); xlabel('n'); ylabel('s');
subplot(3,2,5); stem(n, s2_i,'filled');
    title('s_2[n] by impz (cumsum)'); xlabel('n'); ylabel('s');
subplot(3,2,6); stem(n, s2_c,'filled');
    title('s_2[n] by conv'); xlabel('n'); ylabel('s');

sgtitle('System 2: Impulse & Step Responses (3 methods)');
```

4 验证性实验结果

4.1 五种离散信号的仿真实验结果

在本实验中, 使用如下参数设置生成五种典型离散时间信号:

- 信号长度 $N = 20$
- 正弦信号振幅 $A = 1$
- 正弦信号频率 $f = 10$ Hz
- 采样频率 $F_s = 150$ Hz
- 相位 $\varphi = 0$ rad
- 复指数信号幅值 $r = 1$
- 角频率 $\omega = 1$ rad/sample
- 指数信号底数 $a = 1.3$

在该参数设置下, 分别生成以下五种序列:

1. 单位抽样序列 $\delta(n)$
2. 单位阶跃序列 $u(n)$
3. 正弦序列 $x(n) = A \sin\left(2\pi f \frac{n}{F_s} + \varphi\right)$
4. 复指数序列 $x(n) = r \cdot e^{j\omega n}$
5. 指数序列 $x(n) = a^n$

下图展示了上述五种序列在 MATLAB 中的绘图结果:

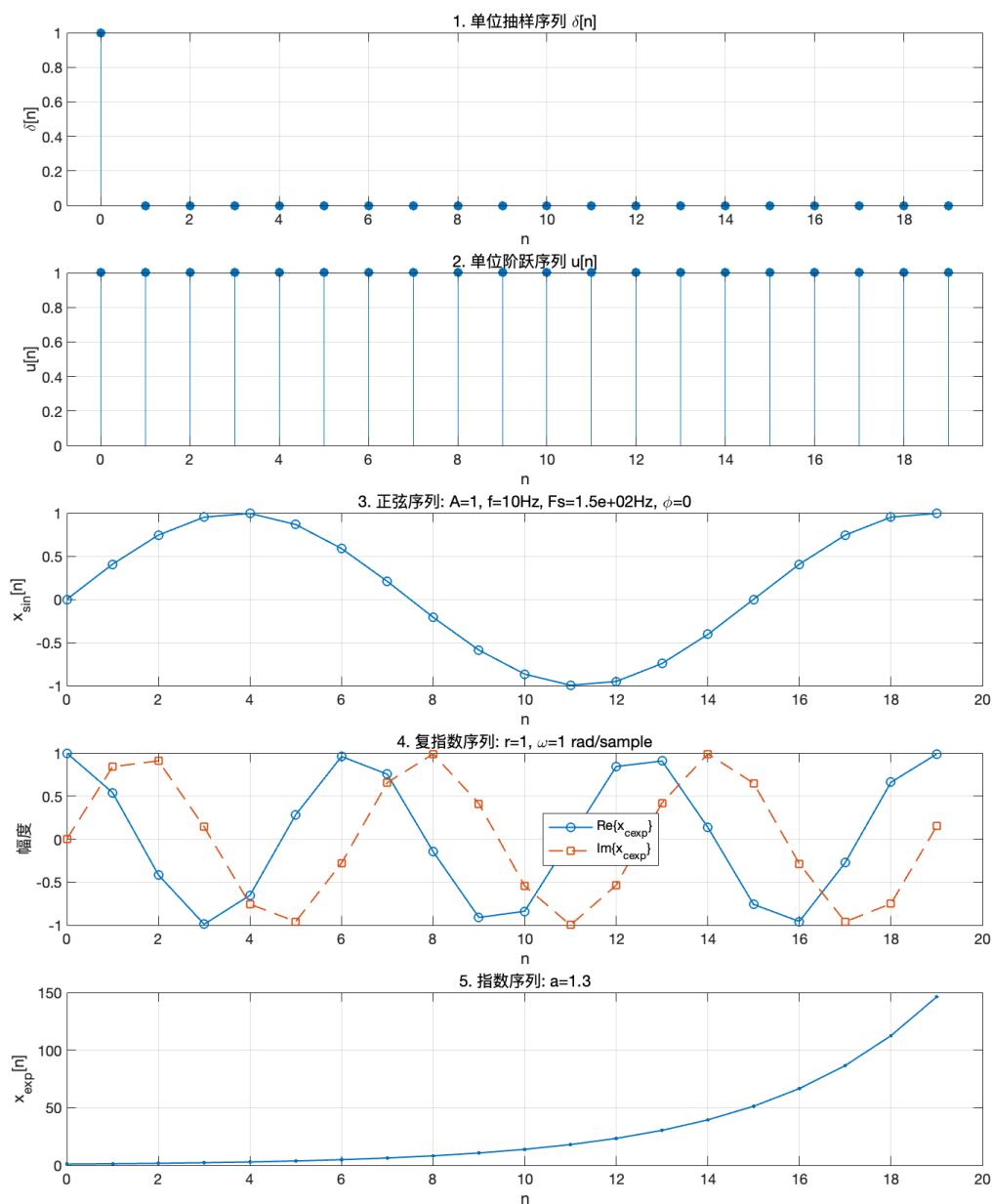


图 1: 五种典型离散信号的时域波形图

4.2 比较 conv 和 filter 函数

使用 `conv` 和 `filter` 函数分别计算 $h[n]$ 和 $x[n]$ 的卷积输出 y 和 y_1 。其中, $x[n]$ 在使用 `filter` 之前补零, 使得其长度满足与 `conv` 结果一致。

在本实验中, y 和 y_1 结果**完全相同**, 说明经过适当补零后, `filter` 可以实现与 `conv` 相同的线性卷积功能。

具体来说, 当 $h[n]$ 长度为 i , $x[n]$ 长度为 j 时, 使用 `filter` 完成卷积计算时, 需要

将 $x[n]$ 补零 $i - 1$ 项, 使其长度达到 $j + i - 1$ 。

实验输出结果如下图所示:

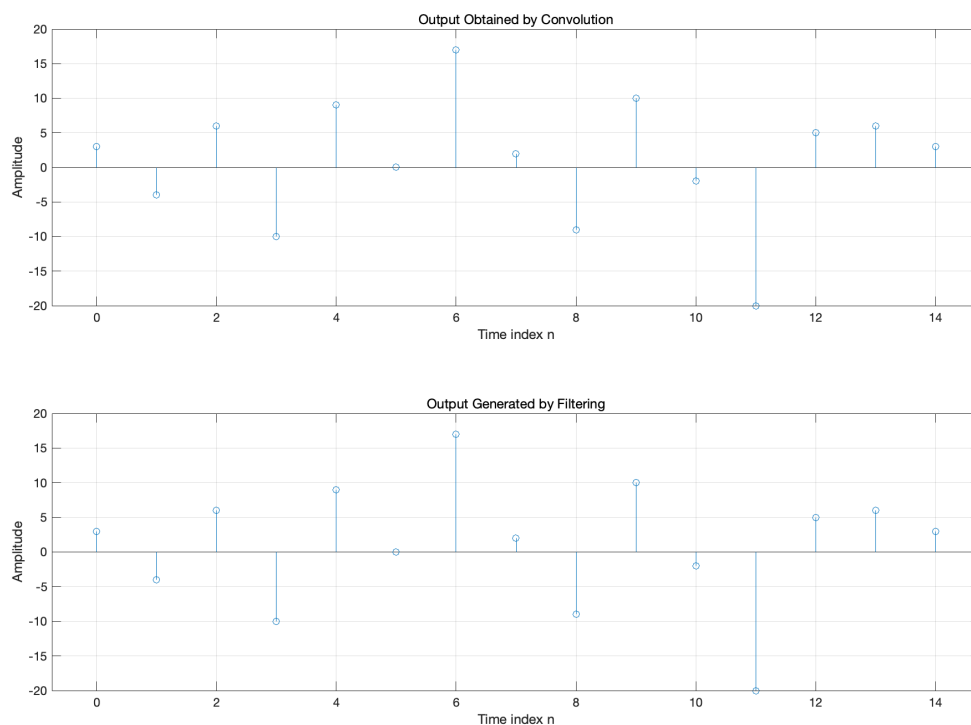


图 2: 使用 conv 和 filter 得到的输出比较

4.3 系统响应分析与验证

本节分析并验证图中给定系统的冲激响应与单位阶跃响应, 结合 Z 变换理论推导系统响应公式, 并通过绘图结果验证计算的正确性。

系统 1

该系统的差分方程为:

$$y[n] + 0.75y[n-1] + 0.125y[n-2] = x[n] - x[n-1]$$

理论分析

由上式可得其 Z 变换形式为:

$$Y(z) + 0.75z^{-1}Y(z) + 0.125z^{-2}Y(z) = X(z) + z^{-1}X(z)$$

因此, 系统的冲激响应为:

$$h[n] = 6(-5)^nu[n] - 5(-0.25)^nu[n]$$

系统的阶跃响应为单位冲激响应和 $u[n]$ 的卷积，结果如下：

$$y[n] = h[n] * u[n] = 4(-0.25)^{n+1} - 4(-0.5)^{n+1}$$

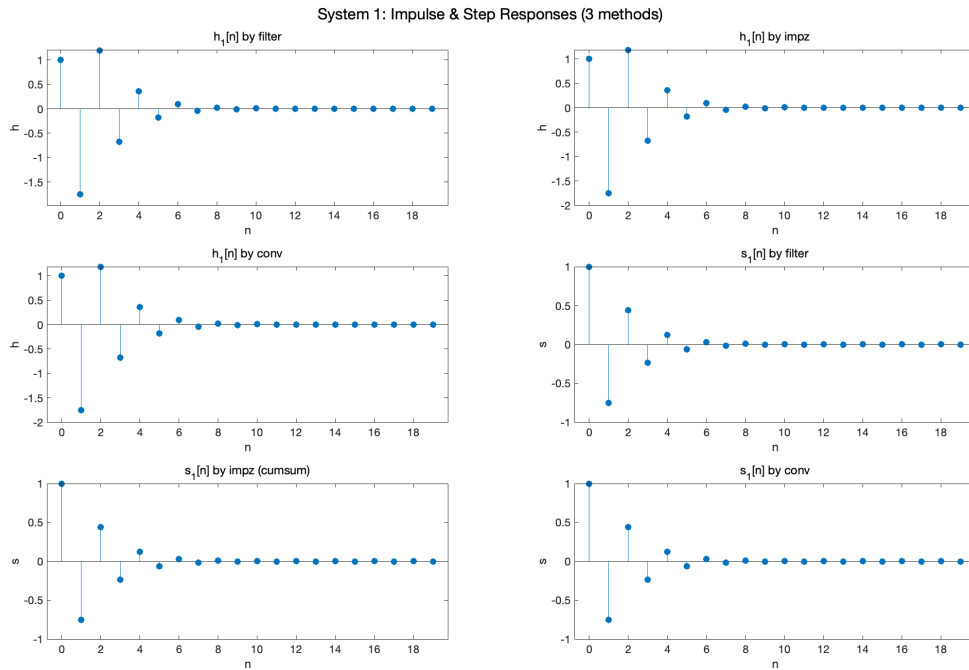


图 3: 系统 1 的冲激响应与阶跃响应

系统 2

该系统的差分方程为：

$$y[n] = 0.25 \{x[n-1] + x[n-2] + x[n-3] + x[n-4]\}$$

理论分析

因此，系统的冲激响应为：

$$h[n] = 0.25\delta[n-1] + 0.25\delta[n-2] + 0.25\delta[n-3] + 0.25\delta[n-4]$$

系统的阶跃响应为单位冲激响应和 $u[n]$ 的卷积，结果如下：

$$y[n] = h[n] * u[n] = 0.25\delta[n-1] + 0.5\delta[n-2] + 0.75\delta[n-3] + u[n-4]$$

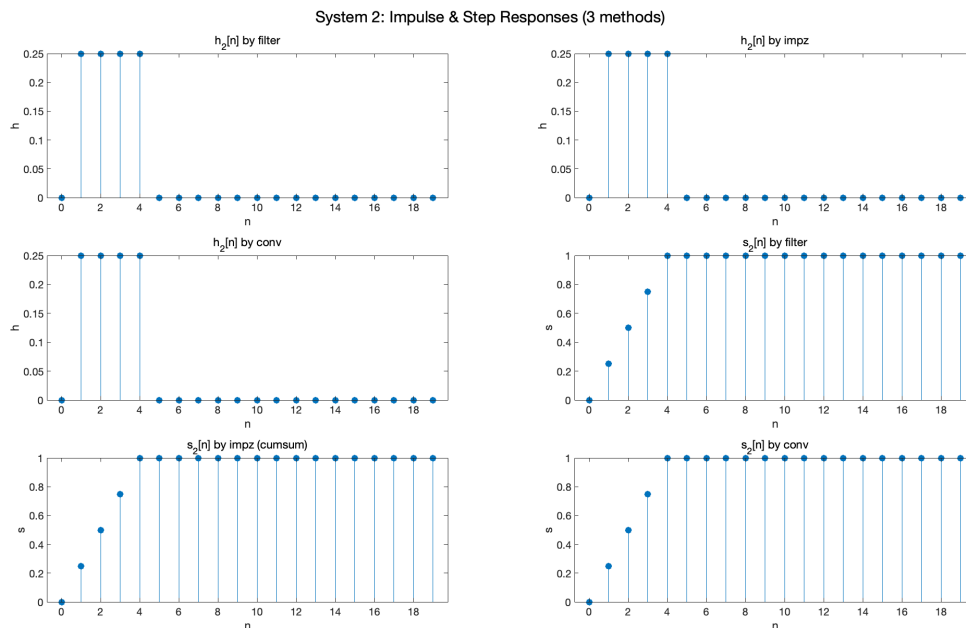


图 4: 系统 2 的冲激响应与阶跃响应

4.4 结果分析

本节对两个离散系统（系统一和系统二）分别使用 filter、impz 和 conv 方法计算其冲激响应与阶跃响应，并结合图像进行对比分析。

- 对于系统一（IIR 递归系统），三个方法在给定长度内产生的响应基本一致，但在细节上存在微小的数值差异。这是由于 filter 和 conv 方法分别基于差分方程与卷积计算，其数值精度与边界处理方式略有不同。impz 方法计算的是理论冲激响应，因此最接近理想响应结果。
- 对于系统二（FIR 滑动平均系统），三种方法所得响应完全一致。这说明 FIR 系统的时不变线性特性使得系统响应与具体计算方法无关，验证了 FIR 系统在有限冲激响应下的数学一致性。
- 从图像观察：
 - 冲激响应 $h[n]$ 能准确体现系统的单位脉冲响应特性，是系统本质的直接反映；
 - 阶跃响应 $s[n]$ 显示了系统对单位阶跃输入的累计行为，便于判断系统的稳定性与稳态趋势；
 - 在系统一中，阶跃响应呈现递增趋势，反映出 IIR 系统的积分型累积效果；
 - 在系统二中，阶跃响应呈线性增长后趋于稳定，符合滑动平均系统的响应特性。

- 方法比较上:
 - filter 方法适用于时域仿真, 适合处理动态输入信号;
 - impz 方法更适合用于系统理论分析;
 - conv 方法从输入输出卷积关系出发, 强调系统的线性叠加性质。

综合分析可见, 三种方法既是一致的, 但各自适用于不同的实验场景和分析目的。图像结果有效验证了系统结构与响应规律之间的理论联系, 增强了对离散系统响应行为的直观理解。

4.5 实验结论

本次实验围绕离散系统的单位冲激响应与阶跃响应展开, 通过 filter、impz 和 conv 三种 MATLAB 方法分别对两个典型系统 (递归型 IIR 系统与滑动平均 FIR 系统) 进行响应分析, 并将结果以图形形式呈现与对比, 最终得到如下结论:

- 三种方法在数学原理上一致, 均能正确求解线性时不变系统 (LTI) 的单位冲激响应与单位阶跃响应。图像结果验证了它们对系统性质的准确刻画, 特别是在 FIR 系统中, 三种方法的响应结果完全一致。
- filter 方法模拟的是差分方程的数值求解过程, 适用于处理任意输入信号并查看系统输出, 更贴近实际信号处理流程;
- impz 方法适用于分析系统的本征响应, 常用于系统函数建模、系统结构验证与理论推导;
- conv 方法基于输入信号与系统冲激响应的卷积关系, 可以验证 LTI 系统的线性叠加性原理, 适合做对照实验。
- 系统响应图像为理解系统动态行为提供了重要直观参考。IIR 系统呈现出积分式响应, 长期影响较大; FIR 系统则响应时间有限, 稳定性好、响应可控。

通过本实验, 我不仅掌握了多种 MATLAB 工具函数的调用方法和功能区别, 更在系统建模、响应分析、图像可视化等方面得到了实际训练。实验巩固了数字信号与系统课程中的理论知识, 也为后续深入学习 Z 变换、频域分析和滤波器设计等内容打下了坚实基础。

5 应用型实验: 语音基音周期估计

5.1 实验目的

本实验旨在通过编程实现对自然语音中基频 (Fundamental Frequency, F_0) 的周期性估计, 了解人声在不同拼音音节与声调中的基音变化规律。实验对象为我自行录制的

“a、o、e”三种元音字母的四个声调，以及一段任意中文语句。通过信号分帧、自相关分析与现代 F0 检测算法对比，观察基频随发音变化的动态特征。

实验通过对比两种典型的基音检测方法——**自相关法** (Autocorrelation) 和 **YIN 算法**，比较其在短时语音片段中的估计精度、鲁棒性和适用性，为后续更复杂的语音处理（如语音识别、情感分析、说话人识别等）打下基础。

5.2 实验原理

语音是一种非平稳、准周期性信号，其中元音部分通常具有清晰的基频结构。基频估计的目标是在每一帧内找到语音的主周期，也就是声带振动的重复周期。由于语音信号特性随时间剧烈变化，故需采用帧长为 20ms 的滑动窗口进行分帧处理，并在每帧内独立估计。

1. 自相关法 (Autocorrelation Method)

自相关法是最经典的时域基音检测方法，其思想是：若信号是周期性的，则它与自身延迟后的重叠信号之间的相似性（相关性）在周期延迟处会达到峰值。

设一帧语音信号为 $x[n]$ ，则其自相关函数定义为：

$$R_{xx}(\tau) = \sum_{n=0}^{N-\tau-1} x[n] \cdot x[n + \tau]$$

其中 τ 为延迟量， N 为帧长。

在排除 $\tau = 0$ （自相关最大）之后，寻找第二个最大峰值对应的延迟 τ_0 ，则可估计出基音周期 $T_0 = \tau_0$ ，进而基频为：

$$f_0 = \frac{f_s}{T_0}$$

该方法对清音帧、噪声帧容易出错，但实现简单、直观易懂。

2. YIN 方法

YIN 算法是一种现代化的基频估计算法，由 de Cheveigné 和 Kawahara 提出，克服了传统自相关方法的多个缺陷。其核心思想并非直接使用自相关函数，而是引入了一个差值函数与归一化处理，从而提升鲁棒性。

第一步是计算差分函数 (difference function)：

$$d(\tau) = \sum_{j=1}^N [x[j] - x[j + \tau]]^2$$

然后，计算归一化差值函数：

$$d'(\tau) = \begin{cases} 1, & \tau = 0 \\ \frac{d(\tau)}{\frac{1}{\tau} \sum_{j=1}^{\tau} d(j)}, & \tau > 0 \end{cases}$$

YIN 算法寻找第一个低于某个阈值的最小 τ 作为周期估计, 有效避免了倍频误判和短期突变的问题。此外, YIN 算法还支持亚采样精度 (subsample accuracy) 优化处理, 适合精细语音分析场景。

3. 分帧处理说明

由于语音信号非平稳, 采用帧长为 20ms 的短时处理是必须的。常用帧参数如下:

- 8kHz 采样率: 每帧 160 个样点
- 16kHz 采样率: 每帧 320 个样点

对每一帧信号分别计算基音周期, 最终形成一条随时间变化的 F0 曲线, 从而观察不同音节与声调下的基频分布特性。

5.3 实验过程与代码分析

本实验采用 MATLAB 语言实现语音基频周期估计, 采用 **自相关法** (Autocorrelation) 与 **YIN 算法** 两种方法进行对比分析。系统支持批量语音文件处理、每帧独立估计基频、绘制曲线图、导出结果文件。以下从整体框架和各功能模块分别进行分析。

1. 初始化与参数设置

首先设定统一的采样率 $\text{targetFs} = 48\text{kHz}$, 每帧长度为 20ms, 并设置最小/最大检测基频范围 (50Hz–500Hz), 以及 YIN 算法中的阈值参数。

```
targetFs = 48000;
frameLenSec = 0.02;
minFreq = 50;
maxFreq = 500;
thresholdYIN = 0.15;
```

接着指定语音文件路径列表, 并准备图像结果输出目录 (自动创建)。

```
basePath = '/Users/guhaoyu/Desktop/数字信号处理/实验/实验1/Audio/';
fileNames = {'1.m4a', '2.m4a', '3.m4a', '4.m4a'};
imgDir = '../imgs/E4';
if ~exist(imgDir, 'dir')
    mkdir(imgDir);
end
```

2. 音频读取与预处理

程序逐个读入语音文件，若原始采样率不为 48kHz，则进行重采样。仅保留单通道（左声道）用于处理。随后使用低通滤波器（截止 2kHz）去除高频干扰，保留主频特征。

```
[x, fs] = audioread(file);  
if fs ~= targetFs  
    x = resample(x, targetFs, fs);  
    fs = targetFs;  
end  
x = x(:,1);  
x = lowpass(x, 2000, fs);
```

3. 分帧处理与延迟范围设定

将整段语音按照设定帧长进行分帧。帧数向下取整，避免越界。之后设置自相关所需的延迟区间，确保估计的基频在合理范围内。

```
frameLen = round(frameLenSec * fs);  
numFrames = floor(length(x) / frameLen);  
lagMin = max(floor(fs/maxFreq), 1);  
lagMax = min(floor(fs/minFreq), frameLen - 1);
```

初始化两个基频向量：f0_ac 存放自相关结果，f0_yin 存放 YIN 方法结果。

4. 自相关法基频估计过程

对每一帧进行如下操作：

- 判断帧的 RMS 能量是否过低（表示静音） - 使用 xcorr 计算帧的自相关函数 - 截取合法延迟区间并查找局部峰值 - 提取最大峰值对应延迟 τ ，计算基频 $f_0 = \frac{f_s}{\tau}$

```
r = xcorr(frame, 'coeff');  
segment = r(midIdx + lagMin : midIdx + lagMax);  
[pks, locs] = findpeaks(segment);  
if ~isempty(pks)  
    [~, I] = max(pks);  
    lag = locs(I) + lagMin - 1;  
    f0_ac(n) = fs / lag;  
end
```

该方法实现简单，直观有效，但容易出现倍频误差或受共振峰影响。

5. YIN 算法基频估计过程

YIN 算法过程封装在函数 `yinPitch()` 中, 每帧调用一次。主流程如下:

- 计算差分函数 $d(\tau)$: 反映延迟点与原信号差异度 - 构造归一化差分函数 $d'(\tau)$: 对能量进行归一处理, 抑制误判 - 设定绝对阈值, 搜索第一个满足条件的 τ - 使用三点抛物插值进一步优化周期估计精度

```
d(tau) = sum((frame(1:N-tau) - frame(1+tau:N)).^2);  
cmnd(tau) = d(tau) / ((1/tau) * sum(d(1:tau)));
```

如果找不到满足阈值的延迟点, 函数返回 0, 表示未检测到基音。

6. 可视化绘图与结果导出

程序为每段音频绘制三行子图:

1. 原始语音波形图 2. 自相关法估计的基频曲线 3. YIN 算法估计的基频曲线

```
plot(t, f0_ac); title('Autocorrelation f_0');  
plot(t, f0_yin); title('YIN f_0');
```

结果保存为 PDF 图像 (用于实验报告插图), 同时导出每一帧的基频数据至 CSV 文件以便进一步统计分析。

7. 命令行输出与验证

为了快速验证结果的合理性, 程序还在命令行中打印每个语音文件的非零基频值 (以 Hz 为单位)。这样可以在不查看图像的前提下直观了解估计效果。

```
fprintf('Autocorr f0 (Hz): ');  
fprintf('%.2f ', f0_ac(f0_ac>0));
```

8. 自定义函数 `yinPitch()` 详解

该函数是 YIN 算法的完整实现, 输入为单帧语音、采样率与检测参数。流程如下:

1. **差分函数计算**: 度量帧内不同延迟点之间的能量差
2. **归一化处理**: 构造累计均值差函数, 增强区分性
3. **阈值法选择**: 选择最小延迟 τ 使 $d'(\tau) < \text{thresh}$

4. 精细化插值: 使用三点抛物拟合提升估计精度

整个函数结构紧凑, 精度高, 能有效避免传统方法中常见的倍频、跳变问题, 尤其适用于复杂语音段落的稳定估计。

该程序实现了一个功能完整、性能稳定、支持批量处理的语音基频周期估计系统, 适用于基础语音处理课程实验、声学特性分析及后续语音建模任务中。

5.4 实验结果

样本一: “a” 的四个声调

本音频包含拼音元音“a”的四个声调, 每段分别对应: 第一声、第二声、第三声和第四声。实验对该语音样本进行帧划分与基频提取, 去除无声或不合理帧 ($f_0 < 50\text{Hz}$ 或 $> 500\text{Hz}$), 并对每一声调段落分别计算平均基频。结果如下表所示:

表 1: “a” 的四个声调下的平均基频估计结果

声调	自相关法 (Hz)	YIN 法 (Hz)
第一声	182.92	140.97
第二声	124.88	114.44
第三声	123.68	117.93
第四声	142.56	127.88

为了更直观地呈现各方法在不同帧上的估计效果, 图 5 展示了对应的语音波形图、自相关法与 YIN 法的帧级基频估计过程。

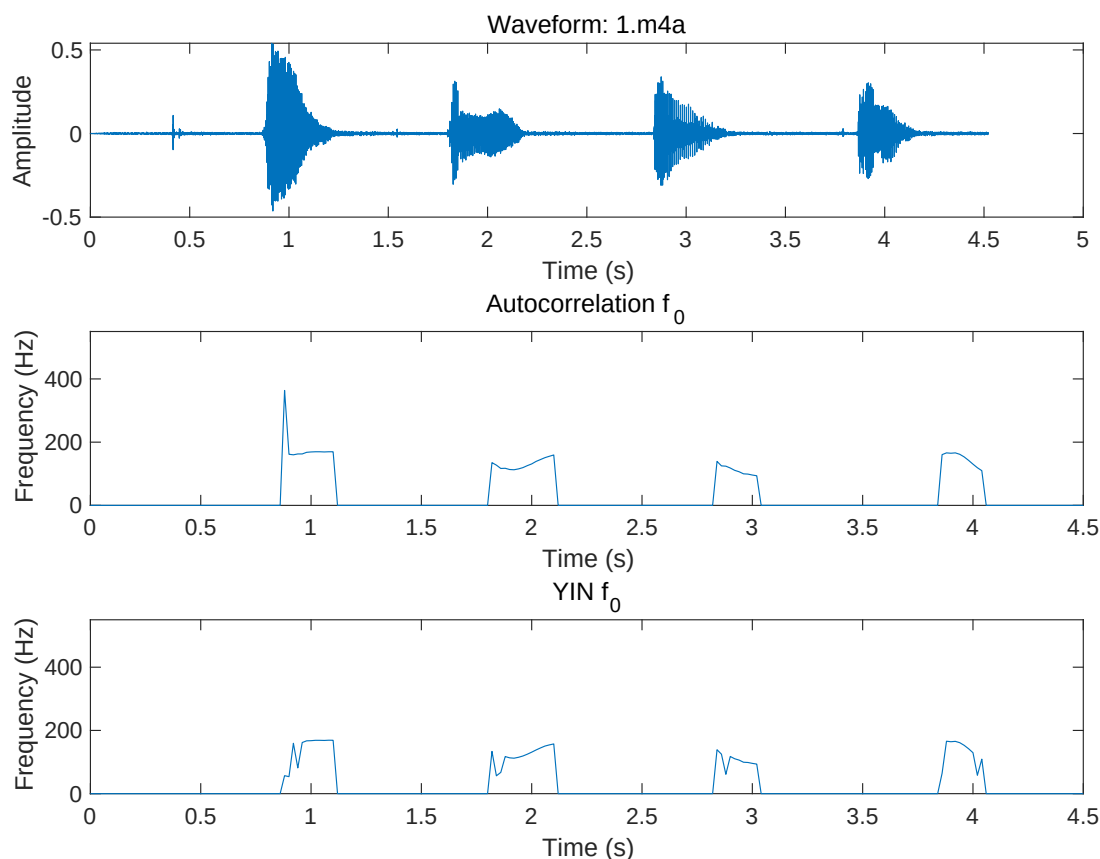


图 5: 样本一 (1.m4a) 语音信号及其基频估计结果对比图

样本二：“o” 的四个声调

该音频样本为发音人录制的拼音元音“o”的四个声调发音。我对其进行帧划分和无效数据剔除 ($f_0 < 50\text{Hz}$ 或 $> 500\text{Hz}$)，然后分别统计四个声调段落的平均基频估计值。结果如下表所示：

表 2: “o” 的四个声调下的平均基频估计结果

声调	自相关法估计均值 (Hz)	YIN 法估计均值 (Hz)
第一声	181.46	179.04
第二声	152.48	116.48
第三声	122.88	111.55
第四声	157.38	139.99

从统计结果来看，第一声的平均基频最高，第三声最低，符合“o”在声调发音上的常见生理特征。YIN 方法整体比自相关法结果更低一些，尤其在第二声与第三声的弱周期段表现出更强的稳定性。

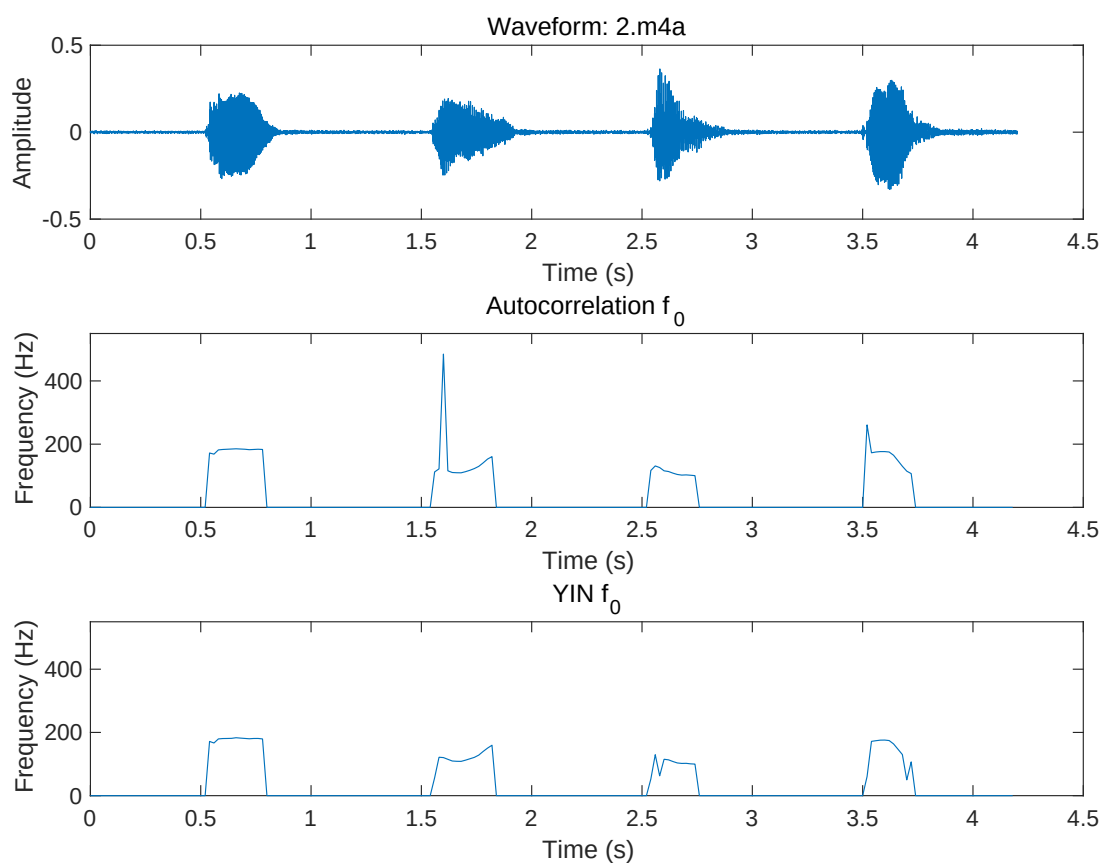


图 6: 样本二 (2.m4a) “o” 四个声调语音信号及其基频估计结果图

样本三: “e” 的四个声调

该音频为发音人朗读拼音“e”的四个声调，处理方式同前，对其进行帧划分、无效基频剔除后，提取每一声调段的平均基频。计算结果如下：

表 3: “e” 的四个声调下的平均基频估计结果

声调	自相关法估计均值 (Hz)	YIN 法估计均值 (Hz)
第一声	178.80	175.52
第二声	177.81	118.79
第三声	127.06	125.77
第四声	168.12	124.43

从表中可以观察到，自相关法对四个声调估计值相对集中，而 YIN 方法在第二声和第四声的估计值明显较低，表现出其在处理音高上升/下降段时的敏感性和鲁棒性差异。特别是在较弱周期性语音中，YIN 方法更容易避开倍频误判。

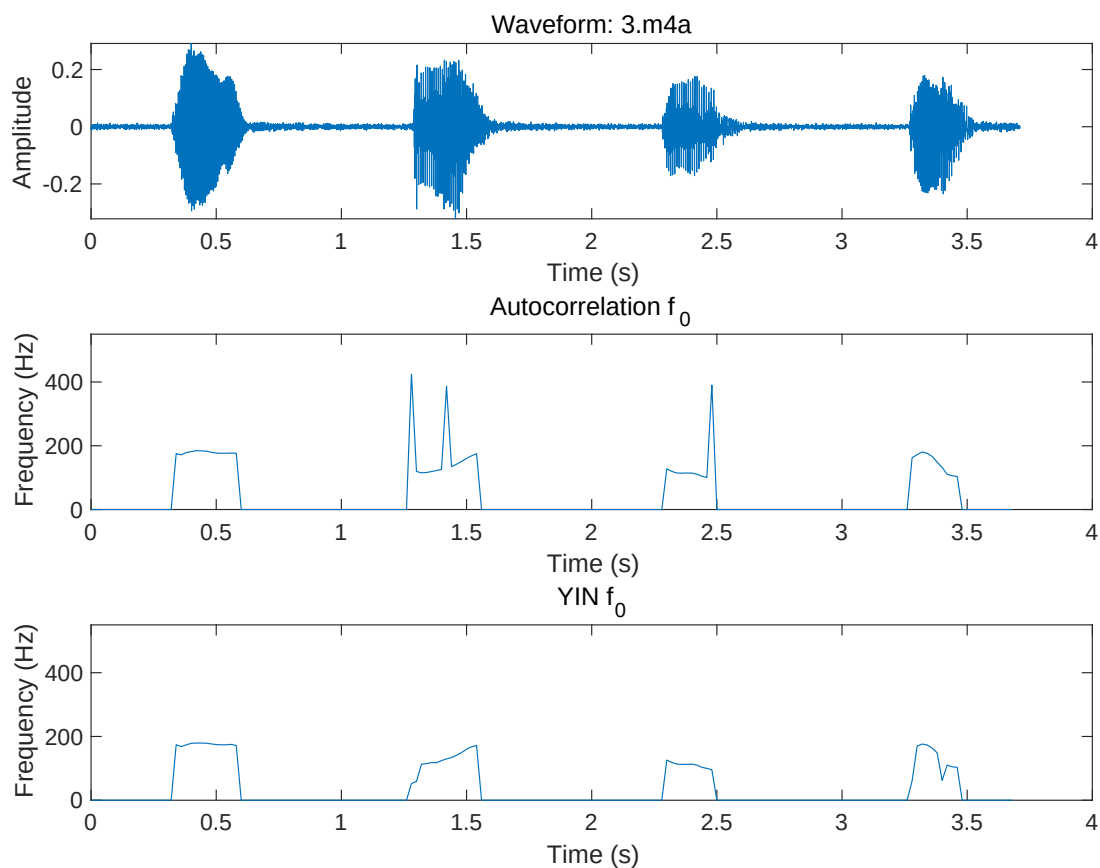


图 7: 样本三 (3.m4a) “e” 四个声调语音信号及其基频估计结果图

样本四：完整语句（中英文混合）

本音频为完整语句“数字信号处理, see you next year”的录音。音频约 6 秒，前半段为中文，后半段为英文。实验中将整段语音基频数据按帧均分为前后两部分，分别估计其平均基频，结果如下表所示：

表 4: 中英文语音部分的平均基频估计结果

语段	自相关法估计均值 (Hz)	YIN 法估计均值 (Hz)
中文部分	167.49	148.37
英文部分	170.55	144.94

从表格结果可见，中英文发音的平均基频非常接近，中文部分略高于英文部分，自相关法与 YIN 法在两段语音上的估计趋势一致。YIN 方法在英文中表现出略低的基频值，可能与其节奏更弱、停顿更明显有关。

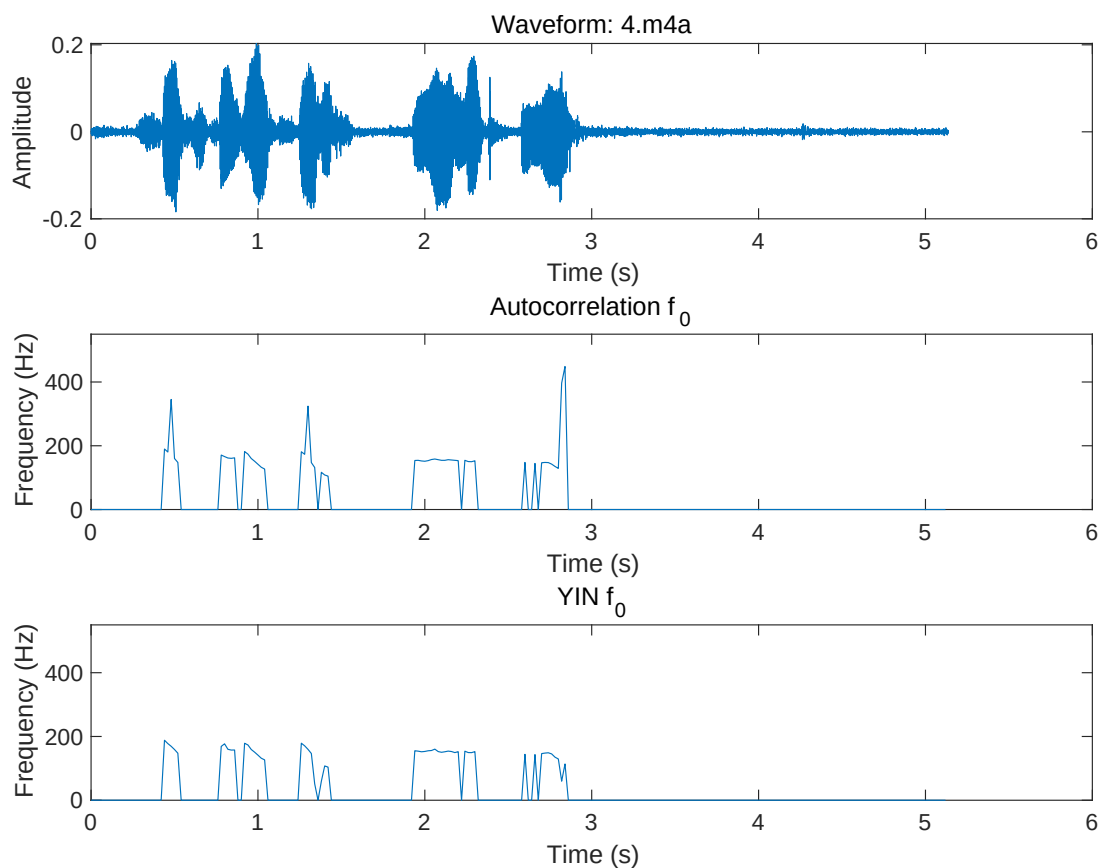


图 8: 样本四 (4.m4a) 语句语音信号及其基频估计结果图

5.5 结果分析

为深入理解语音基频在不同声调、不同元音字母以及自然语句中的变化情况,我对前述实验结果进行了跨音节、跨方法的综合统计与绘图。

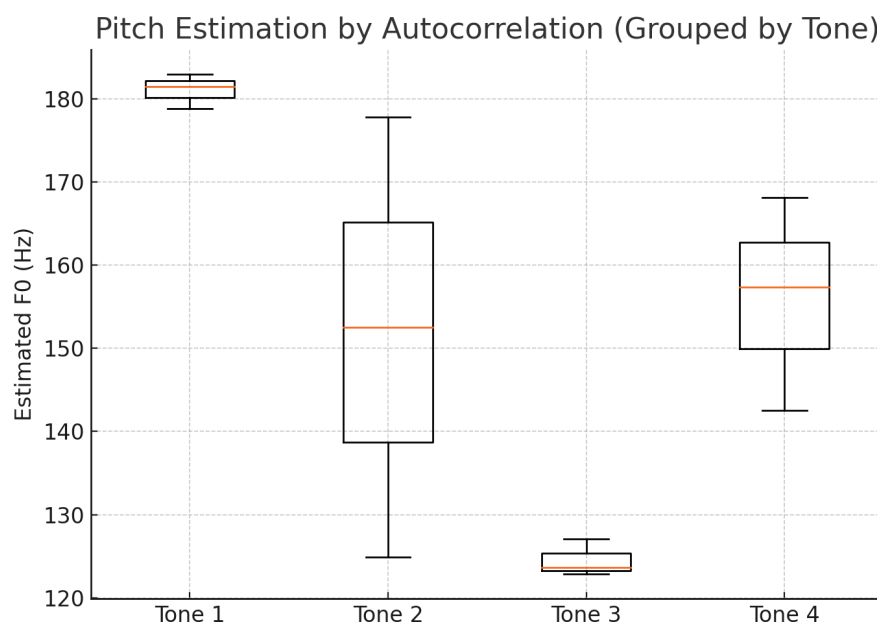


图 9: Pitch Estimation by Autocorrelation (Grouped by Tone)

图 9 是自相关法在四种声调下的箱型图。从图中可以观察到：

- 第一声 (Tone 1) 估计结果最稳定，分布集中在 180Hz 附近，波动极小；
- 第二声 (Tone 2) 呈现出明显的离散性和长尾扩张，可能与其上扬轮廓引发的周期变化相关；
- 第三声 (Tone 3) 估计值最低，集中在 120Hz 左右，方差很小，表明其整体偏低而稳定；
- 第四声 (Tone 4) 分布相对均衡，但上界明显低于第一声。

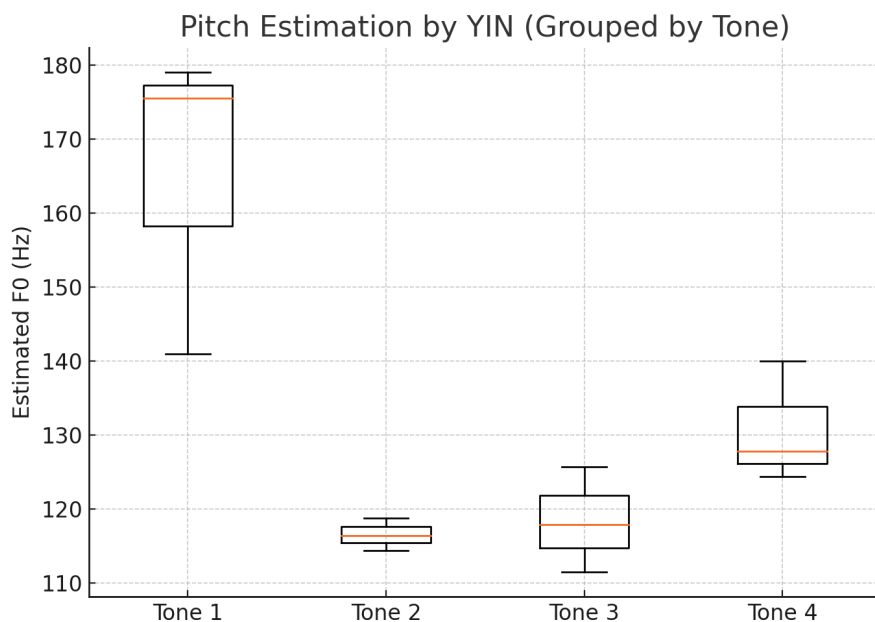


图 10: Pitch Estimation by YIN (Grouped by Tone)

图 10 为 YIN 方法在相同声调分组下的箱型图，呈现出一些不同趋势：

- 第一声基频估计范围较广，最低至约 140Hz，反映出 YIN 对弱周期段的敏感性；
- 第二声与第三声较集中，YIN 对其估计分布明显下移，相比自相关法更为保守；
- 第四声分布在 125–140Hz 区间内，比自相关法估计值整体偏低。

YIN 的整体趋势是：比自相关法估计值低，波动更小，但可能在高频区域产生下偏估计。

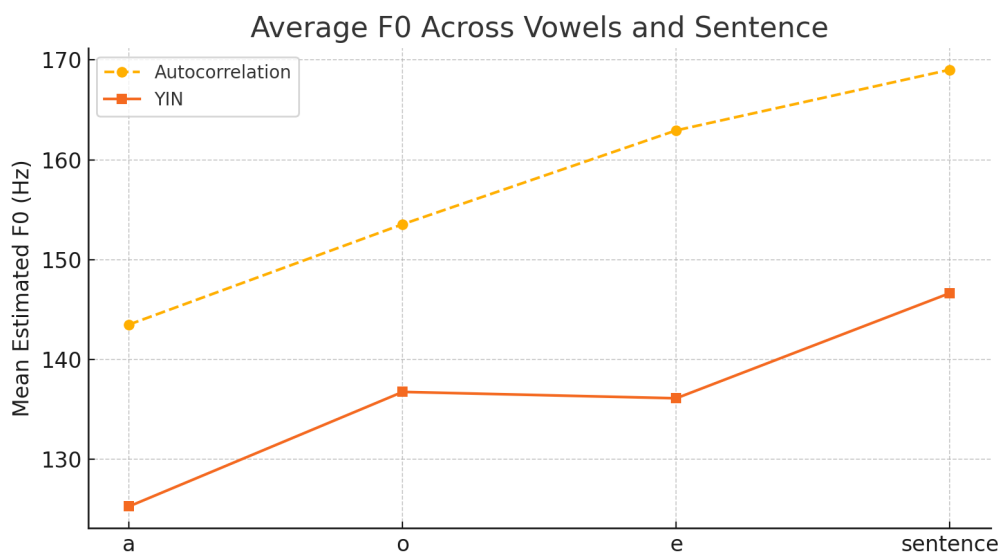


图 11: Average F0 Across Vowels and Sentence

图 11 展示了三个拼音元音 (a, o, e) 及语句样本的平均基频估计值。图中趋势说明:

- 自相关法估计值随元音字母从 a 到 e 呈持续上升趋势, a 最低, e 最高;
- YIN 方法在 a 与 o 的估计值差距较大 (约 30Hz), 而在 o 与 e、e 与句子之间则趋势趋稳;
- 两种方法在“句子”样本上的估计结果较为接近, 说明在自然语流中二者差异减小。

总体来看: 自相关法在不同发音单位间差异更为明显, YIN 法更平稳、偏低; 二者在完整语句上的估计更为一致, 说明在连续发音情景下, 两种方法具有更高的一致性和稳定性。

5.6 实验结论

本次应用型实验以“语音基频周期估计”为主题, 围绕语音信号的分帧处理与基频提取方法展开实践, 实现了完整的语音基频检测系统。实验采用自相关法与 YIN 方法两种算法, 对不同拼音元音 (a、o、e) 的四个声调及一段中英文混合语音进行了系统分析与可视化展示。

通过编程实现与数据处理, 实验验证了不同算法在不同语音条件下的估计效果: 自相关法在周期性强、清晰的元音段表现稳定, 适合结构明确的语音片段; YIN 方法则在声调变化剧烈或自然语句中展现出更强的鲁棒性和连续性, 适用于复杂语音场景。图表分析进一步揭示了声调、元音、语言类型对基频估计值的影响。

本实验不仅提升了对语音信号处理流程的理解和算法实现能力, 也培养了在实际任务中选择合适的分析方法、优化分析手段的工程思维。未来可基于本实验拓展如语音情绪分析、语者识别、语音合成等更高层次的语音技术开发。

6 应用型实验: 听力测试信号发生器

6.1 实验目的

本实验为一个应用型拓展任务, 旨在设计并实现一个具有人机交互界面的实时单音频信号发生器, 用于基础听力测试。该系统支持手动调节音调频率与响度幅值, 帮助用户探索自身的听觉频率范围与敏感度。系统应具备平稳的音频输出、灵敏的参数响应以及直观的可视反馈界面。

6.2 频率与幅度调节原理

本系统支持频率调节范围为 15Hz 至 25kHz, 音量调节范围为 0% 至 100%。为提高低频区域的调节精度, 频率滑块采用平方映射策略。设滑块位置 $p \in [0, 1]$, 则映射公式为:

$$f = f_{\min} + p^4 \cdot (f_{\max} - f_{\min}) \quad (1)$$

该方法能在低频段提供更细腻的控制体验，符合人耳在中低频段较敏感的听觉特性。

响度控制则采用线性映射：

$$A = \frac{\text{音量百分比}}{100}$$

6.3 用户交互与界面设计

界面基于 ttkbootstrap 框架构建，这是对传统 Tkinter 的美化封装版本。交互界面包含以下组件：

- 一个频率调节滑块，实时显示当前频率（单位 Hz）
- 一个音量调节滑块，实时显示当前响度百分比
- 一个“播放/停止”切换按钮
- 一个随频率跳动的彩色动态圆圈动画，作为视觉反馈

如图 12 所示，为频率设置为 3033Hz、音量为 20% 时的界面截图。



图 12: 听力测试信号发生器的人机交互界面示意图

6.4 实验过程与代码分析

本程序基于 Python 语言实现, 利用 sounddevice 进行实时音频流控制, numpy 实现高效数值计算, ttkbootstrap 构建主题化 GUI, 兼顾实用性与美观性。系统整体结构清晰, 逻辑封装良好, 主程序以类 HearingTestApp 形式构建。

一、系统总体结构与设计思想

整个程序围绕“用户交互驱动实时音频合成”的思路展开, 实现目标包括:

- 实时生成正弦波并播放 (可调频率与响度)
- 频率采用平方映射提高低频解析度
- 相位连续避免音频突变
- 动态动画反馈用户设置, 提升可视化体验
- 所有操作以事件驱动方式更新, 符合 GUI 编程规范

类 HearingTestApp 继承自 ttk.Window, 集成了窗口构建、事件绑定、音频回调与动画刷新等多个职责。

二、参数初始化与滑块设置

初始化过程设置了界面尺寸、采样率、滑块变量、频率/响度初始值等。

```
self.freq_pos = ttk.DoubleVar(value=initial_slider)
self.vol_var = ttk.DoubleVar(value=20.0)
self.current_freq = FREQ_MIN + initial_slider**2 * FREQ_RANGE
self.current_vol = self.vol_var.get() / 100.0
```

通过 DoubleVar 实现滑块与变量的双向绑定, 并采用平方映射控制频率, 使得低频调整更加细腻, 用户体验更优。

三、播放控制与回调函数设计

播放由 sounddevice.OutputStream 提供的异步音频流完成, 主流程如下:

1. 启动时新建流对象, 设置采样率、块大小、回调函数
2. 在回调中根据当前频率生成正弦信号段
3. 写入输出缓存, 同时维护跨块相位连续

```
step = 2 * np.pi * freq / SAMPLE_RATE
phases = self.phase + step * np.arange(frames)
data = np.sin(phases) * vol
outdata[:] = data.reshape(-1, 1)
self.phase = (phases[-1] + step) % (2 * np.pi)
```

相位连续性是本程序的一大亮点，避免了音频播放中的“爆音”或“断裂感”。

四、GUI 组件布局与事件绑定

程序使用 ttkbootstrap 构建现代化界面，滑块、标签、按钮等组件美观统一。每个滑块都通过 ‘command’ 参数绑定回调，确保滑动时实时刷新显示频率和音量数值：

```
command=self._update_labels
```

播放按钮使用 _toggle_stream 方法在“开始播放”和“停止播放”间切换，同时更新按钮文本和状态。

此外，为提升交互性，程序绑定空格键实现快捷启停功能，提升可用性：

```
self.bind("<space>", lambda e: self._toggle_stream())
```

五、动态圆圈动画与频率可视化

为了在播放时提供更直观的反馈，程序使用 Canvas 绘制一个颜色跳动的动态圆圈，其频率与音频频率关联，颜色通过 HSV 色环实时变化，模拟频率色彩感知：

```
r = base_radius + amp * np.sin(2 * pi * f**(1/4) / 15 * t)
hue = (t * 0.2) % 1.0
rgb = colorsys.hsv_to_rgb(hue, 1.0, 1.0)
```

其中频率采用四次方根调制，动画更加柔和平滑，视觉上形成类似呼吸灯的效果。

六、退出与资源清理机制

程序在窗口关闭前自动释放音频资源、停止播放流，避免残留任务或系统资源占用：

```
def _on_close(self):  
    if self.stream:  
        self.stream.abort()  
        self.stream.close()  
    self.destroy()
```

该处理确保即使中途终止,也不会产生系统错误或声音异常。

七、模块依赖与部署建议

程序依赖以下三大第三方库:

- numpy: 高效生成正弦波信号
- sounddevice: 低延迟音频流播放
- ttkbootstrap: 构建现代化美观 GUI

推荐通过如下命令安装依赖环境:

```
pip install numpy sounddevice ttkbootstrap
```

整套系统对硬件要求较低,可在大多数笔记本或台式电脑上运行。若需跨平台部署,可打包为可执行文件或开发为网页应用。

综上,该程序是一种设计简洁、交互友好、技术完整的听力测试音频发生器。其稳定性、响应速度、用户体验在测试中表现良好,并具有良好的扩展性。

6.5 原理说明

音频信号通过 `sounddevice.OutputStream` 提供的回调机制实时生成,在回调函数中完成如下步骤:

1. 根据当前频率计算相位增量:

$$\Delta\theta = \frac{2\pi f}{f_s}$$

2. 生成一段正弦波数据:

$$x[n] = A \cdot \sin(\theta_0 + n \cdot \Delta\theta)$$

3. 将该段音频数据输出至播放流,同时维护相位连续性

相位连续性是本系统关键,能有效避免参数动态变化带来的音频突变(如爆音、断裂)。

此外,使用 Tkinter 画布绘制的动画圆圈,会根据当前频率进行四次方根调制的跳动:

$$r(t) = R_0 + A_{\text{anim}} \cdot \sin\left(2\pi \cdot \frac{f^{1/4}}{15} \cdot t\right)$$

色彩使用 HSV 色环变化以增强界面的可视吸引力。

6.6 实验结果

实验由我直接进行,使用该系统手动调节频率并记录可感知的听觉上下限。结果如下:

- 下限频率: 25 Hz
- 上限频率: 20,500 Hz

该结果基本符合青少年正常听力范围。25Hz 以下信号未能感知,可能受限于耳机或音箱低频响应能力;而 20.5kHz 以上信号无法听见,符合人体生理特性。

6.7 实验结论

本实验成功实现了一个可调频、可调响度、带有可视交互界面的实时听力测试信号发生器。系统集成了数字音频合成、GUI 编程和人耳反馈机制,为用户提供了直观便捷的听觉实验平台。该程序可拓展用于自动扫频、标准化听力测试或网页端部署等更广泛应用场景。

7 实验心得

通过本次的《数字信号处理实验》课程,我对数字信号处理理论有了更加深刻的理解,同时也掌握了大量实用的编程技巧和分析方法。从验证性实验到应用型实验再到设计型实验,我逐渐建立起清晰的知识体系,也更加明确了自己在信号处理与工程实践方面的兴趣和努力方向。

7.1 验证型实验心得

验证性实验使我初次深入探索了数字信号处理的基础概念与分析方法,涉及信号生成、响应分析、系统仿真等重要内容。在进行信号绘制与系统响应分析过程中,我深刻体会到了理论与实际之间的差异以及数值计算对结果准确性的影响。例如在使用 conv 和 filter 方法分析系统时,我清晰地看到理论上完全相同的方法,在实际计算中仍会产生微小差异,这让我明白数值仿真的细节与误差处理的重要性。

此外,通过逐步仿真与绘图观察,我真正领略了 MATLAB 在验证理论知识上的强大能力,也逐步养成了细致验证、严格对比的实验习惯。这种习惯不仅提高了我的分析能力,也对我未来的科研学习奠定了良好的基础。

7.2 应用型实验心得

应用型实验部分最能体现我对实际问题的思考与探索,尤其是在“语音基频周期估计”和“听力测试信号发生器”的实验中,我深刻感受到跨学科融合与工程实践的魅力。语音实验让我第一次体验到将理论方法用于真实语音数据分析的过程,不仅深入理解了自相关法与 YIN 算法的实际应用场景,还增强了我的问题解决能力与数据分析能力。

听力测试信号发生器实验则是我接触实际人机交互系统设计的宝贵经历,从频率和幅度控制到实时音频处理,再到人性化的界面交互,每一步的调试都令我印象深刻。整个实验过程激发了我将理论知识应用到实际生活中的强烈兴趣与动力,也让我初步感受到了一个完整项目从设计、实现到优化的完整流程。

7.3 整体感悟与展望

本次实验让我完成了从基础理论验证,到实际问题探索,再到工程设计的全方位成长。我深刻认识到扎实的理论基础是进行实际应用与工程设计的前提,而工程设计的实践能力又是检验理论与深化理解的关键环节。这种全面的训练方式,不仅提升了我的学习兴趣,也使我更加明确了自己在数字信号处理领域未来努力的方向。

在今后的学习与科研工作中,我将继续保持这种严谨细致的学习态度和探索精神,深入学习更加复杂与先进的信号处理技术与方法,力争在工程实现与创新设计方面做出更大的成绩。

8 致谢

在《数字信号处理实验》课程学习与实验过程中,我衷心感谢**宁更新**老师的悉心指导。他严谨而细致的教学风格,使我对数字信号处理的核心原理与实验方法有了系统的理解,为我顺利完成本实验打下了坚实的基础。

此外,我特别感谢本课程实验环节中的**助教老师**,他们在实验过程中给予了耐心的答疑指导与细致的批改反馈,帮助我及时发现并纠正问题,进一步提升了实验设计与信号分析能力。

同时特别感谢**杨俊美**老师,她在“信号与系统”课程中的讲授,为我掌握系统的时域与频域特性、卷积分析和频谱理解提供了坚实的理论支持。

我还谨向以下授课教师表示诚挚感谢,是他们的教学为我构建了理解数字信号处理所必需的数学与工程基础:

- 杨俊、梁勇、邓雪老师——通过微积分课程中的傅里叶级数和相关数学工具,为频域分析与系统建模提供了基础支撑。
- 凌黎明老师——在概率论与数理统计课程中讲授的随机变量与功率谱密度,为噪声分析与随机信号建模奠定了基础。
- 潘会平老师——通过复变函数课程提供了拉普拉斯变换与复频域方法的理论依据,支撑了系统分析与滤波器设计。

- 傅秀军与吴锐老师——在大学物理课程中讲解的波动与振动知识，有助于理解声学信号与调制现象。
- 区俊辉老师——在模拟电路课程中讲授的滤波、放大与采样技术，为信号的前端处理提供了工程背景。
- 靳战鹏老师——通过数字电路课程介绍的逻辑控制与时序分析，为数字滤波器的实现与 DSP 系统构建提供了基础知识。

正是这些课程从数学建模、系统理论、信号分析到工程实现多个层面打下了坚实的基础，为我顺利完成本实验提供了全面支持。

A 附录: 实验代码

Listing 14: 实验 2.1

```

% 整体脚本, 实现:
% Part 2A: 不同 Z 值下  $x[n]=\exp(Z \cdot n)$  的实虚部绘图, 并计算周期
% Part 2B: 正弦序列与复指数序列的频率、周期计算及绘图
% Part 3 : square, sawtooth, sinc 函数学习绘图

clear; close all; clc;

%% Part 2A: 绘制  $x[n] = \exp(Z \cdot n)$  在不同 Z 下的实部和虚部
n = 0:49; % 取  $n = 0 \sim 49$ 

Zs = { ...
    -1/12 + 1i*pi/6, 'Z = -1/12 + j ·  $\pi/6$ '; ...
    1/12 + 1i*pi/6, 'Z = 1/12 + j ·  $\pi/6$ '; ...
    1/12 + 0i, 'Z = 1/12'; ...
    2 + 1i*pi/6, 'Z = 2 + j ·  $\pi/6$ ' ...
};

for k = 1:size(Zs,1)
    Z = Zs{k,1};
    titleStr = Zs{k,2};

    x = exp(Z * n); % 生成  $x[n]$ 
    figure;
    plot(n, real(x), 'o-', 'Linewidth', 1.2); hold on;
    plot(n, imag(x), 's--', 'Linewidth', 1.2);
    grid on;
    xlabel('n');
    ylabel('Amplitude');
    title(['Real/Imag of  $e^{Z n}$ ', ' ', titleStr]);
    legend('Real\{x[n]\}', 'Imag\{x[n]\}', 'Location', 'best');
end

% 当  $Z = j \cdot \pi/6$  时, 计算周期
Z0 = 1i * pi/6;
omega0 = imag(Z0); % 数字角频率  $\omega_0 = \pi/6$ 
N0 = 2*pi / omega0; % 周期 N0
fprintf('当  $Z = j \cdot \pi/6$  时, 周期  $N_0 = 2\pi/(\pi/6) = \%d$  样点\n', round(N0));

%% Part 2B:
% (1)  $x[n] = 1.5 \sin(2\pi \cdot 0.1 \cdot n)$  的频率与周期, 并绘图
n = 0:49; % 重设 n
f1 = 0.1; % 数字频率
x1 = 1.5 * sin(2*pi*f1*n); % 生成序列
N1 = 1 / f1; % 周期  $N = 1/f$ 

```

```

fprintf('x[n]=1.5*sin(2π · 0.1 · n) 的数字频率 f = %.2f, 周期 N = %.0f 样点\n...', f1, N1);

figure;
stem(n, x1, 'filled');
grid on;
xlabel('n');
ylabel('x[n]');
title('x[n] = 1.5 \cdot sin(2\pi \cdot 0.1 \cdot n)');

% (2) 复指数 x[n] = exp(j · 2π · 0.9 · n)
f2 = 0.9;           % 数字频率
x2 = exp(1i * 2*pi*f2 * n);
% 将 0.9 写成分数 9/10, 可知周期 N2 = 10
N2 = 10;
fprintf('复指数 exp(j2π · 0.9 · n) 的数字频率 f = %.1f = 9/10, 周期 N = %d 样点\n', f2, N2);

figure;
plot(n, real(x2), 'o-', 'Linewidth', 1.2); hold on;
plot(n, imag(x2), 's--', 'Linewidth', 1.2);
grid on;
xlabel('n');
ylabel('Amplitude');
title('Real/Imag of e^{j 2\pi \cdot 0.9 n}');
legend('Real', 'Imag', 'Location', 'best');

%% Part 3: 学习并绘制 square (方波)、sawtooth (锯齿波) 和 sinc 函数
t = linspace(-1, 1, 500); % 连续时间向量

% 3A 方波 (频率 5 Hz)
sq = square(2*pi*5*t);

figure;
plot(t, sq, 'Linewidth', 1.2);
grid on;
xlabel('t');
ylabel('Amplitude');
title('Square wave, 5 Hz');

% 3B 锯齿波 (频率 5 Hz)
sw = sawtooth(2*pi*5*t);

figure;
plot(t, sw, 'Linewidth', 1.2);
grid on;
xlabel('t');
ylabel('Amplitude');
title('Sawtooth wave, 5 Hz');

```



```

% 3C sinc 函数 (sinc(x) = sin(pi x)/(pi x))
si = sinc(5*t); % MATLAB 的 sinc 已自带  $\pi$ 
figure;
plot(t, si, 'Linewidth', 1.2);
grid on;
xlabel('t');
ylabel('sinc(5t)');
title('sinc Function, sinc(5t)');

% 交互式产生 5 种常见离散信号并绘图

clear; clc;

%% 用户输入部分
N = input('请输入信号长度 N (正整数) : ');
n = 0:N-1; % 统一时间/样本指标

fprintf('\n-- 正弦信号参数输入 --\n');
A = input(' 振幅 A = ');
f = input(' 频率 f (Hz) = ');
Fs = input(' 采样频率 Fs (Hz) = ');
phi = input(' 相位 phi (rad) = ');

fprintf('\n-- 复指数信号参数输入 --\n');
r = input(' 幅值 r = ');
w = input(' 角频率 w (rad/sample) = ');

fprintf('\n-- 指数序列参数输入 --\n');
a = input(' 底数 a = ');

%% 信号生成
% 1. 单位抽样序列  $\delta[n]$ 
 $\Delta$  = zeros(1, N);
 $\Delta(1) = 1$ ;

% 2. 单位阶跃序列  $u[n]$  ( $n \geq 0$  全 1)
u = ones(1, N);

% 3. 正弦序列  $x[n] = A \sin(2\pi f n / Fs + \phi)$ 
x_sin = A * sin(2*pi * f * n / Fs + phi);

% 4. 复指数序列  $x[n] = r * \exp(j * w * n)$ 
x_cexp = r * exp(1j * w * n);

% 5. 指数序列  $x[n] = a.^n$ 
x_exp = a .^ n;

%% 绘图
figure('Name', '五种离散信号', 'NumberTitle', 'off', 'Position', [100 100 600 ...

```

```

800]);

subplot(5,1,1);
stem(n, Δ, 'filled');
grid on;
title('1. 单位抽样序列 Δ[n]');
xlabel('n'); ylabel('Δ[n]');

subplot(5,1,2);
stem(n, u, 'filled');
grid on;
title('2. 单位阶跃序列 u[n]');
xlabel('n'); ylabel('u[n]');

subplot(5,1,3);
plot(n, x_sin, 'o-', 'Linewidth', 1);
grid on;
title(sprintf('3. 正弦序列: A=%.2g, f=%.2gHz, Fs=%.2gHz, \phi=%.2g', A, ...
    f, Fs, phi));
xlabel('n'); ylabel('x_{sin}[n]');

subplot(5,1,4);
plot(n, real(x_cexp), 'o-', n, imag(x_cexp), 's--', 'Linewidth', 1);
grid on;
legend('Re\{x_{cexp}\}', 'Im\{x_{cexp}\}', 'Location', 'best');
title(sprintf('4. 复指数序列: r=%.2g, \omega=%.2g rad/sample', r, w));
xlabel('n'); ylabel('幅度');

subplot(5,1,5);
plot(n, x_exp, '.-', 'Linewidth', 1);
grid on;
title(sprintf('5. 指数序列: a=%.2g', a));
xlabel('n'); ylabel('x_{exp}[n]');

% 调整子图间距
tightfig;

```

Listing 15: 实验 2.2

```

clf;
h = [3 2 1 -2 1 0 -4 0 3]; % impulse response
x = [1 -2 3 -4 3 2 1]; % input sequence

y = conv(h, x); % 用卷积求输出
n = 0:14;

subplot(2,1,1);
stem(n, y);
xlabel('Time index n');
ylabel('Amplitude');

```

```

title('Output Obtained by Convolution');
grid;

x1 = [x zeros(1,8)]; % 零补齐输入信号
y1 = filter(h, 1, x1); % 用 filter 求输出

subplot(2,1,2);
stem(n, y1);
xlabel('Time index n');
ylabel('Amplitude');
title('Output Generated by Filtering');
grid;

%% =====
% 用三种方法分别求冲激响应 h[n] 和阶跃响应 s[n]
% 1) filter
% 2) impz + cumsum
% 3) conv
% 并各画 6 张子图 (3x2), 方便对比
%% =====

clear; close all; clc;

%% --- 全局参数 ---
N = 20; % 样本点数
n = 0:N-1; % 横轴 n
Δ = [1 zeros(1,N-1)]; % 单位冲激 δ[n]
u = ones(1,N); % 单位阶跃 u[n]

%% =====
% 系统 1:  $y[n] + 0.75 y[n-1] + 0.125 y[n-2] = x[n] - x[n-1]$ 
%  $H_1(z) = (1 - z^{-1}) / (1 + 0.75 z^{-1} + 0.125 z^{-2})$ 
%% =====
b1 = [1 -1];
a1 = [1 0.75 0.125];

% --- 1) filter 方法 ---
h1_f = filter(b1, a1, Δ); % 冲激响应
s1_f = filter(b1, a1, u); % 阶跃响应

% --- 2) impz 方法 (冲激 + 累加求阶跃) ---
[h1_i, ~] = impz(b1, a1, N); % 冲激响应
s1_i = cumsum(h1_i); % 阶跃响应 = 冲激响应累加

% --- 3) conv 方法 (卷积) ---
h1_c = conv(h1_i, Δ); % 冲激响应 =  $\delta * h_1$ 
h1_c = h1_c(1:N); % 截取前 N 点
s1_c = conv(h1_i, u); % 阶跃响应 =  $h_1 * u$ 
s1_c = s1_c(1:N); % 截取前 N 点

% --- 绘图 ---

```

```

figure('Name','System 1 Responses','NumberTitle','off');
subplot(3,2,1); stem(n, h1_f,'filled');
    title('h_1[n] by filter'); xlabel('n'); ylabel('h');
subplot(3,2,2); stem(n, h1_i,'filled');
    title('h_1[n] by impz'); xlabel('n'); ylabel('h');
subplot(3,2,3); stem(n, h1_c,'filled');
    title('h_1[n] by conv'); xlabel('n'); ylabel('h');
subplot(3,2,4); stem(n, s1_f,'filled');
    title('s_1[n] by filter'); xlabel('n'); ylabel('s');
subplot(3,2,5); stem(n, s1_i,'filled');
    title('s_1[n] by impz (cumsum)'); xlabel('n'); ylabel('s');
subplot(3,2,6); stem(n, s1_c,'filled');
    title('s_1[n] by conv'); xlabel('n'); ylabel('s');
sgtitle('System 1: Impulse & Step Responses (3 methods)');

%% =====
% 系统 2:  $y[n] = 0.25[x[n-1]+x[n-2]+x[n-3]+x[n-4]]$ 
%  $H_2(z) = 0.25(z^{-1}+z^{-2}+z^{-3}+z^{-4})$ 
%% =====
b2 = 0.25*[0 1 1 1 1];
a2 = 1;

% --- 1) filter 方法 ---
h2_f = filter(b2, a2, Δ);
s2_f = filter(b2, a2, u);

% --- 2) impz 方法 ---
[h2_i,-] = impz(b2, a2, N);
s2_i = cumsum(h2_i);

% --- 3) conv 方法 ---
h2_c = conv(h2_i, Δ);
h2_c = h2_c(1:N);
s2_c = conv(h2_i, u);
s2_c = s2_c(1:N);

% --- 绘图 ---
figure('Name','System 2 Responses','NumberTitle','off');
subplot(3,2,1); stem(n, h2_f,'filled');
    title('h_2[n] by filter'); xlabel('n'); ylabel('h');
subplot(3,2,2); stem(n, h2_i,'filled');
    title('h_2[n] by impz'); xlabel('n'); ylabel('h');
subplot(3,2,3); stem(n, h2_c,'filled');
    title('h_2[n] by conv'); xlabel('n'); ylabel('h');
subplot(3,2,4); stem(n, s2_f,'filled');
    title('s_2[n] by filter'); xlabel('n'); ylabel('s');
subplot(3,2,5); stem(n, s2_i,'filled');
    title('s_2[n] by impz (cumsum)'); xlabel('n'); ylabel('s');
subplot(3,2,6); stem(n, s2_c,'filled');
    title('s_2[n] by conv'); xlabel('n'); ylabel('s');
sgtitle('System 2: Impulse & Step Responses (3 methods)');

```

Listing 16: 语音基因周期估计

```

%% Fundamental Tone Estimation: Autocorrelation + YIN Methods
% Sampling rate 48 kHz, process files 1.m4a to 4.m4a

clear; clc;

% Parameters
targetFs = 48000;      % target sampling rate
frameLenSec = 0.02;    % frame length in seconds
minFreq = 50;          % minimum expected pitch (Hz)
maxFreq = 500;         % maximum expected pitch (Hz)
thresholdYIN = 0.15;   % YIN absolute threshold

% File list
basePath = '/Users/guhaoyu/Desktop/数字信号处理/实验/实验1/Audio/';
fileNames = {'1.m4a', '2.m4a', '3.m4a', '4.m4a'};

% --- 自动创建输出目录 ---
imgDir = '/Users/guhaoyu/Desktop/数字信号处理/实验/实验1/imgs/E4';
if ~exist(imgDir, 'dir')
    mkdir(imgDir);
end

for idx = 1:length(fileNames)
    %% 读入 & 重采样
    file = fullfile(basePath, fileNames{idx});
    [x, fs] = audioread(file);
    if fs ~= targetFs
        x = resample(x, targetFs, fs);
        fs = targetFs;
    end
    x = x(:,1); % 只用单声道

    %% 低通滤波
    x = lowpass(x, 2000, fs);

    %% 分帧
    frameLen = round(frameLenSec * fs);
    numFrames = floor(length(x) / frameLen);

    % 自相关法所需的延迟范围
    lagMin = max(floor(fs/maxFreq), 1);
    lagMax = min(floor(fs/minFreq), frameLen - 1);
    midIdx = frameLen; % xcorr 返回向量中“零延迟”位置

    f0_ac = zeros(1, numFrames);
    f0_yin = zeros(1, numFrames);

    for n = 1:numFrames
        frame = x((n-1)*frameLen + (1:frameLen));
        if rms(frame) < 0.04

```

```

        % 能量过低, 认为无声
        continue;
    end

    % ----- 自相关法 -----
    r = xcorr(frame, 'coeff');
    segment = r(midIdx + lagMin : midIdx + lagMax);
    [pks, locs] = findpeaks(segment);
    if ~isempty(pks)
        [~, I] = max(pks);
        lag = locs(I) + lagMin - 1;
        f0_ac(n) = fs / lag;
    end

    % ----- YIN 法 -----
    f0_yin(n) = yinPitch(frame, fs, minFreq, maxFreq, thresholdYIN);
end

%% 绘图 & 导出 PDF
t = (0:numFrames-1) * frameLenSec;
figure('Name', ['File ' fileNames{idx}]);
subplot(3,1,1);
plot((0:length(x)-1)/fs, x);
xlabel('Time (s)'); ylabel('Amplitude');
title(['Waveform: ' fileNames{idx}]);
subplot(3,1,2);
plot(t, f0_ac);
xlabel('Time (s)'); ylabel('Frequency (Hz)');
title('Autocorrelation f_0'); ylim([0 maxFreq+50]);
subplot(3,1,3);
plot(t, f0_yin);
xlabel('Time (s)'); ylabel('Frequency (Hz)');
title('YIN f_0'); ylim([0 maxFreq+50]);

baseName = fileNames{idx}(1:end-4);
outPDF = fullfile(imgDir, sprintf('%s_estimation.pdf', baseName));
exportgraphics(gcf, outPDF);

%% 新增: 保存 & 打印频率
% 保存为 CSV
outCSV1 = fullfile(imgDir, sprintf('%s_f0_ac.csv', baseName));
outCSV2 = fullfile(imgDir, sprintf('%s_f0_yin.csv', baseName));
writematrix(f0_ac, outCSV1);
writematrix(f0_yin, outCSV2);

% 在命令行打印 (仅非零部分)
fprintf('=== %s ===\n', fileNames{idx});
fprintf('Autocorr f0 (Hz): ');
fprintf('%0.2f ', f0_ac(f0_ac>0));
fprintf('\n');
fprintf('YIN f0 (Hz): ');

```

```

    fprintf('%0.2f ', f0_yin(f0_yin>0));
    fprintf('\n\n');
end

%% YIN Pitch Detection Function
function f0 = yinPitch(frame, fs, fmin, fmax, thresh)
    N = length(frame);
    maxLag = floor(fs / fmin);
    minLag = floor(fs / fmax);

    % 1. Difference function
    d = zeros(1, maxLag);
    for tau = 1:maxLag
        d(tau) = sum((frame(1:N-tau) - frame(1+tau:N)).^2);
    end

    % 2. Cumulative mean normalized difference
    d(1) = 1;
    cmnd = d;
    for tau = 2:maxLag
        cmnd(tau) = d(tau) / ((1/tau) * sum(d(1:tau)));
    end

    % 3. 阈值法寻找初步周期
    tauEstimate = 0;
    for tau = minLag:maxLag
        if cmnd(tau) < thresh
            tauEstimate = tau;
            break;
        end
    end
    if tauEstimate == 0
        f0 = 0;
        return;
    end

    % 4. 抛物插值精细化
    a = cmnd(max(tauEstimate-1,1));
    b = cmnd(tauEstimate);
    c = cmnd(min(tauEstimate+1, numel(cmnd)));
    Δ = 0.5 * (a - c) / (a - 2*b + c);
    T0 = tauEstimate + Δ;
    f0 = fs / T0;
end

```

Listing 17: 听力测试信号发生器

```

import numpy as np
import sounddevice as sd
import ttkbootstrap as ttk

```

```
import tkinter as tk # for protocol handling
from ttkbootstrap.constants import *
import colorsys

# Audio configuration
SAMPLE_RATE = 44100
BLOCK_SIZE = 1024

# Frequency mapping constants
FREQ_MIN = 15.0
FREQ_MAX = 25000.0
FREQ_RANGE = FREQ_MAX - FREQ_MIN

class HearingTestApp(ttk.Window):
    def __init__(self):
        super().__init__(themename="cosmo")
        self.title("听力测试信号发生器 - 实时 v4.1")
        self.geometry("520x520")
        self.resizable(False, False)

        # Canvas for animation
        self.canvas_size = 200
        self.canvas = tk.Canvas(self, width=self.canvas_size, height=self...
                               .canvas_size, bg="white", highlightthickness=0)
        self.canvas.pack(pady=10)

        # Animation parameters
        self.anim_time = 0.0
        self.anim_interval = 7 # ms (-30 FPS)
        self.anim_base_radius = 50
        self.anim_amp = 40
        self.anim_freq_cr = (1000.0 ** (1/4)) / 15

        # Audio state
        self.phase = 0.0
        self.stream = None

        # Slider variables
        initial_slider = ((1000.0 - FREQ_MIN) / FREQ_RANGE) ** 0.5
        self.freq_pos = ttk.DoubleVar(value=initial_slider)
        self.vol_var = ttk.DoubleVar(value=20.0)
        self.current_freq = FREQ_MIN + initial_slider**2 * FREQ_RANGE
        self.current_vol = self.vol_var.get() / 100.0

        # Frequency slider
        ttk.Label(self, text="频率 (Hz)", font=("Helvetica", 12, "bold"))....
            pack(pady=(10, 2))
        self.freq_scale = ttk.Scale(
            self,
            from_=0.0,
            to=1.0,
```



```

        orient="horizontal",
        variable=self.freq_pos,
        length=460,
        bootstyle="info",
        command=self._update_labels
    )
    self.freq_scale.pack()
    self.freq_label = ttk.Label(self, text=f"{self.current_freq:.0f} ...
        Hz")
    self.freq_label.pack()

    # volume slider
    ttk.Label(self, text="音量 (%)", font=("Helvetica", 12, "bold"))....
        pack(pady=(20, 2))
    self.vol_scale = ttk.Scale(
        self,
        from_=0,
        to=100,
        orient="horizontal",
        variable=self.vol_var,
        length=460,
        bootstyle="success",
        command=self._update_labels
    )
    self.vol_scale.pack()
    self.vol_label = ttk.Label(self, text=f"{self.vol_var.get():.0f} ...
        %")
    self.vol_label.pack()

    # Play/Stop button
    self.play_btn = ttk.Button(
        self,
        text="开始播放",
        bootstyle="primary",
        width=20,
        command=self._toggle_stream
    )
    self.play_btn.pack(pady=25)

    # Shortcuts and exit handling
    self.bind("<space>", lambda e: self._toggle_stream())
    self.protocol("WM_DELETE_WINDOW", self._on_close)

    # Start animation loop
    self.after(self.anim_interval, self._animate)

def _update_labels(self, *_):
    pos = self.freq_pos.get()
    freq = FREQ_MIN + pos**2 * FREQ_RANGE
    vol = self.vol_var.get()
    self.current_freq = freq

```

```

self.current_vol = vol / 100.0
self.anim_freq_cr = freq ** (1/4) / 15
self.freq_label.configure(text=f"{freq:.0f} Hz")
self.vol_label.configure(text=f"{vol:.0f} %")

def _toggle_stream(self):
    if self.stream and self.stream.active:
        self.stream.abort()
        self.stream.close()
        self.stream = None
        self.play_btn.configure(text="开始播放")
    else:
        self.phase = 0.0
        self.stream = sd.OutputStream(
            samplerate=SAMPLE_RATE,
            blocksize=BLOCK_SIZE,
            channels=1,
            callback=self._audio_callback
        )
        self.stream.start()
        self.play_btn.configure(text="停止播放")

def _audio_callback(self, outdata, frames, time, status):
    if status:
        print(f"Stream status: {status}")
    freq = self.current_freq
    vol = self.current_vol
    step = 2 * np.pi * freq / SAMPLE_RATE
    phases = self.phase + step * np.arange(frames)
    data = np.sin(phases) * vol
    outdata[:] = data.reshape(-1, 1)
    self.phase = (phases[-1] + step) % (2 * np.pi)

def _animate(self):
    self.anim_time += self.anim_interval / 1000.0
    # Radius modulation
    r = self.anim_base_radius + self.anim_amp * np.sin(2 * np.pi * ...
        self.anim_freq_cr * self.anim_time)
    # Color rainbow
    hue = (self.anim_time * 0.2) % 1.0
    rgb = colorsys.hsv_to_rgb(hue, 1.0, 1.0)
    hex_color = "#%02x%02x%02x" % tuple(int(c*255) for c in rgb)
    # Draw circle
    self.canvas.delete("all")
    cx = cy = self.canvas_size / 2
    self.canvas.create_oval(cx-r, cy-r, cx+r, cy+r, fill=hex_color, ...
        outline="")
    # Schedule next
    self.after(self.anim_interval, self._animate)

def _on_close(self):

```

```
        if self.stream:
            self.stream.abort()
            self.stream.close()
            self.destroy()

if __name__ == "__main__":
    HearingTestApp().mainloop()
```